Verifiable Inevitability of Computation

Cem Sel
Independent Researcher
cem@adpriva.com

Abstract

Many distributed systems require mechanisms to impose computational cost on digital actions in order to mitigate abuse, regulate access, and discourage large-scale automation. Existing approaches—including proof-of-work, client puzzles, verifiable delay functions, and CAPTCHA-based systems—typically rely on probabilistic cost, synchronized time assumptions, trusted hardware, or actor classification.

In this work, we introduce the notion of *verifiable inevitability of computation*: the ability for a verifier to determine whether a claimed action could have existed without performing a prescribed amount of computational effort, without relying on identity, clocks, or behavioral interpretation.

We formalize this notion and present the Computational Effort Layer We define a security model, analyze security in a sequential hash-chain / random-oracle-style query model, and analyze efficiency and deployment tradeoffs. Our results demonstrate that inevitability of computation can be enforced as a protocol-layer primitive independent of identity and time.

1 Introduction

Distributed systems frequently require mechanisms to impose computational cost on digital actions. Such mechanisms are essential for mitigating spam, limiting abusive automation, resisting denial-of-service attacks, and discouraging large-scale Sybil amplification. Despite decades of research, the Internet still lacks a native primitive for enforcing computational cost on individual actions in a manner that is independent of identity, synchronized time, and behavioral classification.

Existing approaches rely on assumptions that are increasingly brittle. Identity-based rate limits fail in adversarial environments where identities

can be generated at negligible cost. Time-based mechanisms require trusted clocks or synchronized delay assumptions. Proof-of-work systems impose probabilistic cost, introducing variance and allowing adversaries to benefit from lucky outcomes. CAPTCHA-based mechanisms rely on actor classification and degrade both privacy and accessibility.

These limitations motivate a more fundamental question:

> Can a verifier determine whether a digital action could have existed without performing a prescribed amount of computation, without relying on identity, time synchronization, or semantic interpretation?

This work introduces a new cryptographic perspective on this question. Rather than attempting to identify actors, measure elapsed time, or classify behavior, we focus on the *inevitability* of computation. Informally, an action is inevitable if no efficient adversary could have produced the corresponding proof artifact without executing a required sequence of computational steps.

Verifiable inevitability. We formalize this notion as *verifiable inevitability of computation*: the ability for a verifier to confirm that a claimed action could not have existed without performing a specified amount of sequential computation. Crucially, inevitability differs from delay. The goal is not to prove that a certain amount of wall-clock time has elapsed, but rather that a dependency chain of computation could not have been bypassed.

This distinction separates our work from verifiable delay functions (VDFs), which aim to measure elapsed time under sequentiality assumptions. In contrast, inevitability concerns whether computation was unavoidable, independent of when or how quickly it was executed.

Our approach. We present a protocol construction, termed the *Computational Effort Layer (CEL)*, that realizes verifiable inevitability under standard cryptographic assumptions. The construction enforces deterministic per-action cost through sequential dependency, producing compact receipts whose validity implies unavoidable computation.

The protocol admits two verification regimes. In the first, verification is performed by direct recomputation of the dependency chain. In the second, verification is achieved through succinct proofs of correct execution, allowing verification cost to remain constant even for large computational depth. This dual-mode structure enables practical deployment across both low-adversary and high-adversary environments.

Contributions. This paper makes the following contributions:

- We introduce the notion of *verifiable inevitability of computation* as a cryptographic property distinct from proof-of-work and verifiable delay.
- We formalize a security model capturing adversarial attempts to shortcut sequential computation.
- We present a protocol construction achieving deterministic, identity-free cost enforcement without reliance on synchronized time.
- We prove that any adversary capable of forging a valid receipt without performing the required computation yields a reduction that breaks the preimage resistance of cryptographic hash functions.
- We analyze efficiency, discuss deployment tradeoffs, and compare the construction to existing cost-enforcement mechanisms.

2 Related Work

The problem of enforcing computational cost in adversarial environments has been studied extensively across cryptography and distributed systems. Prior work may be broadly categorized into pricing mechanisms, proof-of-work systems, client puzzles, verifiable delay functions, and succinct proof systems. We review each line of work and position our contribution accordingly.

2.1 Pricing via Processing

The foundational idea of imposing computational cost to deter abuse was introduced by Dwork and Naor [1], who proposed *pricing via processing* as a mechanism to combat electronic junk mail. Their work established the principle that forcing senders to perform computation can economically deter large-scale abuse.

Subsequent systems adopted this idea in various contexts; however, early proposals lacked formal security models and provided no guarantees beyond heuristic cost amplification.

2.2 Proof-of-Work Systems

Proof-of-work (PoW) mechanisms, most notably Hashcash [2], require participants to solve computational puzzles whose difficulty can be adjusted. PoW systems impose expected computational cost through probabilistic search.

While PoW has proven effective in blockchain systems, it exhibits several limitations in the context of per-action enforcement. First, cost is probabilistic rather than deterministic, allowing adversaries to benefit from variance. Second, PoW provides no assurance that a particular action required a minimum amount of computation—only that it was sampled from a distribution. Finally, PoW receipts do not encode unavoidable dependency structure.

CEL differs fundamentally by enforcing deterministic sequential dependency rather than probabilistic work.

2.3 Client Puzzles

Client puzzles [3] apply computational challenges to mitigate denial-of-service attacks by forcing clients to expend effort before consuming server resources. Numerous variants have been proposed in networking and transport-layer protocols.

Client puzzles typically assume online interaction, challenge freshness, and server-issued puzzles. They also lack transferable receipts and provide no cryptographic notion of inevitability once interaction completes.

In contrast, CEL produces self-contained receipts whose validity is independent of online challenge-response interaction.

2.4 Verifiable Delay Functions

Verifiable delay functions (VDFs) [4, 5] enforce sequential computation intended to approximate elapsed time. Recent work has focused on improved constructions, proof efficiency, and hardware assumptions.

VDFs address a different objective from our work. Their goal is to verify that a certain amount of time has elapsed under sequentiality assumptions. CEL instead seeks to verify that a sequence of computation was inevitable, independent of wall-clock time.
We note that the underlying sequentiality assumption is the same one used in VDF and iterated-hashing client-puzzle constructions; our contribution is the framing and protocol layer (context/epoch binding, receipts) rather than a new hardness assumption.

In particular, CEL does not attempt to approximate time, does not rely on time-based soundness assumptions, and does not require globally agreed delay parameters.

2.5 Succinct Proof Systems

Succinct non-interactive arguments of knowledge (SNARKs and STARKs) [6] enable verification of arbitrary computation with sublinear overhead. Recent advances have dramatically reduced prover and verifier costs, enabling deployment in practical systems.

CEL leverages succinct proofs as an optional verification mechanism but does not rely on any specific proof system. Instead, succinct proofs serve as a compression layer for verifying unavoidable computation, rather than as the core enforcement mechanism.

2.6 Anti-Sybil and Abuse-Resistance Mechanisms

A wide body of work explores Sybil resistance through resource-based approaches, including computational, memory, and bandwidth constraints. Recent systems incorporate hybrid techniques combining identity, stake, or hardware assumptions.

CEL intentionally avoids actor classification, identity, and stake. Its objective is orthogonal: enforcing unavoidable computation at the protocol layer without assumptions about who performs it.

Positioning. In summary, prior work enforces cost through probabilistic effort, elapsed time, or actor classification. To the best of our knowledge, no prior system formalizes or enforces the notion of *verifiable inevitability of computation* as a distinct cryptographic primitive.

Our work fills this gap by separating sequential dependency from time and defining a security model in which the inability to shortcut computation constitutes the central guarantee.

3 Preliminaries

3.1 Notation

Let $\lambda \in \mathbb{N}$ denote the security parameter.

For a probabilistic algorithm $\mathcal{A}$, we write

$$\Pr[\mathcal{A}(\lambda) = 1]$$

to denote the probability that $\mathcal{A}$ outputs 1 when executed with security parameter λ.

We write $x \parallel y$ for the concatenation of binary strings x and y. All encodings are assumed to be prefix-free unless stated otherwise.

An adversary is modeled as a probabilistic polynomial-time (PPT) algorithm.

3.2 Cryptographic Hash Functions

Let

$$\mathcal{H} : \{0,1\}^* \to \{0,1\}^\lambda$$

be a cryptographic hash function.

We assume $\mathcal{H}$ satisfies standard *preimage resistance*:

Definition 1 (Preimage Resistance)**.** *A hash function $\mathcal{H}$ is preimage resistant if for all PPT adversaries $\mathcal{A}$,*

$$\Pr\left[x \leftarrow \mathcal{A}(y) \ : \ \mathcal{H}(x) = y\right] \leq \mathrm{negl}(\lambda),$$

where y is chosen uniformly at random from $\{0,1\}^\lambda$.

For the sequential-cost argument, we model hash evaluation as an idealized sequential query process. Preimage resistance is necessary, but preimage resistance alone does not establish unpredictability of H's output to an adversary lacking the corresponding input; Theorem 1 below is stated relative to the random-oracle model for this reason.

3.3 Sequential Dependency

We formalize the notion of non-skippable computation via sequential dependency.

Definition 2 (Sequential Dependency)**.** *A computation is sequentially dependent if the input to step i cannot be computed without knowledge of the output of step $i-1$.*

Sequential dependency ensures that no algorithm can evaluate later steps without executing all prior steps, except with negligible probability under standard cryptographic assumptions.

3.4 Receipts and Verification

A *receipt* is a finite string intended to attest that a prescribed computation was executed.

A verification algorithm Verify takes as input a receipt R and outputs either accept or reject.

A receipt is considered *valid* if it is accepted by Verify under the specified policy parameters.

3.5 Adversarial Model

The adversary is allowed to:

- perform arbitrary polynomial-time preprocessing,
- adaptively choose inputs,
- parallelize computation freely,
- store unbounded intermediate state.

The adversary is not assumed to be bounded by hardware class, memory model, or interaction constraints.

The only restriction is polynomial-time execution in the security parameter.

3.6 Inevitability of Computation

We now formalize the central concept of this work.

Definition 3 (Inevitable Computation)**.** *A receipt attests to inevitable computation if no PPT adversary can produce a valid receipt without executing all prescribed sequentially dependent steps, except with negligible probability.*

This definition does not depend on elapsed time, identity, or behavioral classification. It captures only whether the computation could have been skipped.

4 The CEL Construction

In this section we formally define the Computational Effort Layer (CEL) construction.

4.1 Public Parameters

The construction operates under the following public parameters:

- Security parameter λ
- Hash function $\mathcal{H} : \{0,1\}^* \to \{0,1\}^\lambda$

All parties are assumed to have access to $\mathcal{H}$.

4.2 Policy

A *policy* is a tuple

$$P = (n),$$

where $n \in \mathbb{N}$ denotes the required depth of sequential computation.

The policy determines the computational cost required to produce a valid receipt.

4.3 Context Binding

Each computation is bound to a contextual string

$$Ctx \in \{0,1\}^*,$$

which may encode application-specific data such as domain identifiers, action labels, or resource identifiers.

CEL treats Ctx as an opaque string; its semantics are not interpreted by the protocol.

4.4 Epoch Identifier

An epoch identifier

$$E \in \{0,1\}^*$$

is used to scope receipts to bounded validity windows.

The construction does not assume synchronized clocks. Epochs may be derived from local policy, block heights, or application-defined intervals.

4.5 Sequential Assembly

Given policy P = (n), epoch E, and context Ctx, the prover computes a sequential assembly as follows.

Initialization:
s0 = H(frame(P) || frame(E) || frame(Ctx))

Assembly (for i = 1, ..., n):
e_i = H(s_{i-1} || uint64be(i))
s_i = H(s_{i-1} || e_i)

The value s_n is referred to as the assembly root. The v0 implementation uses length-prefixed framing and uint64 big-endian step indices.

4.6 Receipt

A receipt is defined as

$$R = (P, E, Ctx, s_n, n).$$

Intuitively, R attests that the prover has executed a sequential computation of depth n bound to (P, E, Ctx).

4.7 Verification

Given receipt R, the verifier recomputes the assembly starting from
s0 = H(frame(P) || frame(E) || frame(Ctx))

and accepts if and only if the recomputed value equals s_n.

4.8 Optional Succinct Verification

The construction permits an optional succinct proof π attesting knowledge of the intermediate states

$$(s_1, \ldots, s_{n-1}, \epsilon_1, \ldots, \epsilon_n)$$

such that the above assembly equations hold.

When provided, verification may be performed in sublinear or constant time depending on the proof system used.

The security analysis is independent of the specific succinct argument system.

5 Security Model

We formalize the security of the Computational Effort Layer through a game-based definition.

Intuitively, security requires that no efficient adversary can produce a valid receipt without performing the prescribed sequential computation.

5.1 Adversarial Model

The adversary $\mathcal{A}$ is modeled as a probabilistic polynomial-time (PPT) algorithm.

The adversary is allowed to:

- choose arbitrary context strings Ctx
- choose epoch identifiers E
- adaptively attempt to generate receipts

The adversary is not restricted in parallelism or computational resources, except for polynomial-time bounds.

5.2 Verification Predicate

Let $\mathsf{Verify}(P, E, Ctx, s_n, n)$ be the deterministic verification algorithm defined in Section 4.

Verification outputs 1 if and only if recomputation of the assembly yields s_n.

5.3 Sequential Assembly Game

We define the following security game.

Game 1 (Sequential Assembly Game)**.**

1. *The challenger samples public parameters and fixes a policy* $P = (n)$.
2. *The adversary* $\mathcal{A}$ *outputs* (E, Ctx).
3. *The adversary outputs a candidate receipt*

$$R = (P, E, Ctx, s_n, n).$$

4. *The adversary wins if:*
   (a) $\mathsf{Verify}(P, E, Ctx, s_n, n) = 1$, *and*
   (b) $\mathcal{A}$ *did not evaluate the full sequential assembly of depth* n.

Formally, we model A's interaction with H as a sequence of oracle queries partitioned into rounds, where queries within a round may be made in parallel but each round may depend on the outputs of all prior rounds. Condition 4(b) means A completed fewer than n such rounds before outputting R.

5.4 Security Definition
Definition 4 (Inevitability Security). The CEL construction is said to satisfy inevitability of computation if for all PPT adversaries A, Pr [A wins Game 1] ? negl(?).
This definition captures the core security goal: producing a valid receipt must be computationally inevitable.

6 Security Analysis

We prove that producing a valid CEL receipt without executing the required sequential computation is computationally infeasible under standard cryptographic assumptions.

6.1 Sequential Dependency

We first formalize the dependency structure induced by the construction.

Definition 5 (Sequential Assembly)**.** *A sequential assembly of depth n is a sequence of states*

$$(s_0, s_1, \ldots, s_n)$$

such that for all i in {1, ..., n}:

e_i = H(s_{i-1} || uint64be(i))
s_i = H(s_{i-1} || e_i)

Each state uniquely depends on the immediately preceding state.

6.2 Unskippability Property

Lemma 1 (Unskippable Dependency)**.** *Given s_i, computing s_{i+1} without knowledge of s_i requires computing a preimage of $\mathcal{H}$ with non-negligible probability.*

Proof. By Definition 5,
e_{i+1} = H(s_i || uint64be(i+1))
s_{i+1} = H(s_i || e_{i+1})

Any algorithm computing s_{i+1} without knowing s_i must correctly guess a value that is pseudorandom in the random oracle model (since s_{i+1} = H(s_i || e_{i+1}) and both s_i and e_{i+1} are required oracle inputs). Under the random-oracle assumption on H, this occurs with probability at most 2^{-?}, which is negligible.

6.3 No-Skip Lemma

Lemma 2 (No-Skip Property)**.** *For any PPT adversary, producing s_n without computing all intermediate states $(s_0, \ldots, s_{n-1})$ occurs with negligible probability.*

Proof. Assume an adversary skips some index $j < n$.
Then either:

1. the adversary computes s_{j+1} without knowing s_j, contradicting Lemma 1, or

2. the adversary computes a preimage of $\mathcal{H}$.

Both events occur with negligible probability under the preimage resistance of $\mathcal{H}$. □

6.4 Main Security Argument

Theorem 1 (Inevitability Security Argument). Model H as a random oracle. Then the CEL construction satisfies inevitability of computation (Definition 4) in the round-based query model of Section 5.3.

Proof. Suppose for contradiction that a PPT adversary $\mathcal{A}$ wins the Sequential Assembly Game with non-negligible probability.

Then $\mathcal{A}$ outputs a valid receipt (P, E, Ctx, s_n, n) such that verification succeeds.

Verification implies the existence of a valid sequential assembly consistent with Definition 5.

By Lemma 2, computing s_n without computing all intermediate states is negligible.

Therefore $\mathcal{A}$ must have performed the full sequential computation, contradicting the winning condition of the game.

Hence, the probability that any PPT adversary wins the game is negligible. □

6.5 Discussion

The security argument relies on sequential hash-chain evaluation in an idealized query model, with preimage resistance as a necessary underlying property. It does not assume:
- trusted hardware
- timing assumptions
- identity binding
- classification of actors

Sequential inevitability arises from the structure of chained evaluations. The argument is stated in the random-oracle model rather than from preimage resistance alone; see the footnote in Section 3.2 for why the weaker assumption does not suffice.

7 Implementation and Evaluation

We implemented a reference prototype of the CEL construction to evaluate its practical feasibility and performance characteristics.

7.1 Implementation Details

The public v0.1 reference implementation is written in dependency-free JavaScript for Node.js 18 or newer, with a standalone browser implementation using WebCrypto. The Node implementation uses the built-in crypto module and supports SHA-256 and SHA-512.

The implementation follows the construction described in Section 4.5 and performs sequential assembly without parallelization. No GPU acceleration or specialized hardware was used.

7.2 Experimental Methodology

For each experiment, we measure:

- total assembly time,
- verification time under Mode A (recomputation),
- throughput under repeated receipt generation.

Each measurement is averaged over 100 independent runs.

All reported times correspond to single-threaded execution to reflect worst-case sequential cost.

7.3 Performance Results

Table 1 reports observed assembly and verification costs for increasing depth values.

The results confirm linear scaling with respect to assembly depth, as predicted by the construction.

Depth	Create Receipt	Direct Verify
10,000	~22 ms	~22 ms
100,000	~219 ms	~220 ms
1,000,000	~2,240 ms	~2,280 ms

Table 1: Sequential assembly and Mode A verification performance. These are local development measurements and should be treated as rough guidance, not portable benchmarks.

7.4 Comparison with Existing Mechanisms

We qualitatively compare CEL with alternative cost-enforcement mechanisms.

- **Proof-of-Work** provides probabilistic cost and allows variance-based shortcutting.
- **VDFs** enforce elapsed time but require strong timing assumptions.
- **CAPTCHAs** rely on classification and external trust assumptions.

In contrast, CEL enforces deterministic sequential dependency without relying on identity, clocks, or behavioral assumptions.

7.5 Mode B Verification

While our prototype focuses on Mode A verification, the construction could potentially be extended with succinct verification using SNARK or STARK systems. In such deployments, the prover incurs the full assembly cost while the verifier performs constant or logarithmic work. Mode B is not implemented in v0.1 and remains future research.

7.6 Scalability Discussion

CEL prevents reuse or amortization of one receipt across properly bound contexts, but it does not prevent an attacker from generating many independent receipts in parallel on many machines.

8 Limitations and Future Work

While the CEL construction establishes verifiable inevitability of computation under standard cryptographic assumptions, it is not a universal solution to all abuse-prevention or fairness problems.

8.1 Scope Limitations

CEL intentionally avoids addressing several dimensions that are orthogonal to computational inevitability:

- **Identity and attribution.** CEL does not identify actors, distinguish users, or provide reputation.
- **Intent and semantics.** The protocol verifies only that computation occurred, not whether an action was meaningful, correct, or authorized.
- **Economic fairness.** CEL enforces cost but does not guarantee equitable outcomes across heterogeneous hardware environments.

As a result, CEL should be viewed as a low-level primitive rather than a complete abuse-prevention system.

8.2 Hardware Asymmetry

Although CEL enforces unavoidable sequential dependency, different hardware platforms may still exhibit performance asymmetries.

In particular, specialized hardware such as ASICs or custom accelerators may reduce wall-clock latency, though not the asymptotic per-receipt cost.

Future work may explore memory-hard or hybrid constructions to further limit specialized hardware advantages.

8.3 Mode B Proof Systems

Our evaluation focuses on direct verification (Mode A).

The efficiency of succinct verification (Mode B) depends on the chosen proof system and circuit representation.

Future work includes:

- optimized arithmetic circuit encodings,
- proof aggregation techniques,
- recursive proof composition for batched verification.

8.4 Adaptive Policies

The current formulation assumes a static policy defining assembly depth.

An interesting direction is adaptive policy selection based on observed system load, threat level, or resource availability.

Such policies could dynamically adjust required effort without modifying the core verification logic.

8.5 Broader Applications

Beyond abuse mitigation, potential applications include:

- rate-limited decentralized messaging,
- identity-free access control,
- verifiable resource contribution claims,
- cryptographic throttling for public APIs.

Exploring these domains in depth remains future work.
These directions are left for future work.

9 Conclusion

In this work, we introduced the notion of verifiable inevitability of computation, a property enabling a verifier to determine whether a digital action could have existed without performing a prescribed amount of computational effort.

We presented the Computational Effort Layer (CEL), a protocol construction that enforces deterministic, per-action computational cost without reliance on identity, trusted hardware, synchronized clocks, or behavioral classification.

We formalized a security model capturing adversarial shortcut attempts and analyzed sequential security in a hash-chain / random-oracle-style query model. We further analyzed efficiency tradeoffs between direct recomputation and succinct verification, demonstrating that inevitability can be enforced at the protocol layer with bounded verification cost.

We believe that inevitability-oriented primitives occupy a complementary position between proof-of-work systems and time-based constructions, offering a new design axis for abuse-resistant and resource-aware distributed systems.

We hope this work stimulates further research into identity-free cost enforcement mechanisms and their applications in decentralized and adversarial environments.

References

[1] C. Dwork and M. Naor. Pricing via processing. CRYPTO 1992.

[2] A. Back. Hashcash. 1997.

[3] A. Juels and J. Brainard. Client puzzles. NDSS 1999.

[4] D. Boneh et al. Verifiable delay functions. CRYPTO 2018.

[5] K. Pietrzak. Simple verifiable delay functions. ITCS 2019.

[6] N. Bitansky et al. The SNARK papers. 2018.